Kolejka priorytetowa

Kolejka priorytetowa jest to struktura danych, w której dostęp do danych jest możliwy tylko do pierwszego elementu, a dane w niej są uporządkowane nierosnąco (lub niemalejąco). Kolejka priorytetowa jest teoretycznie określana mianem FIFO (First In First Out), czyli pierwszy wchodzący element jest pierwszym wychodzącym, chociaż praktycznie odnosi się to do najwyższego „priorytetu”

Taki rodzaj kolejki ma bardzo powszechne zastosowanie — od zarządzania wiadomościami w aplikacjach internetowych, aż po jądro systemu operacyjnego.

Zauważ różnicę w interfejsie w porównaniu do prostych kolejek: kolejka priorytetowa nie udostępnia dostępu do ostatnio wstawionego elementu (odpowiednik back w kolejce). Zamiast tego elementy są pobierane przez kolejkę priorytetową, a ona zwraca tylko najwyższy element.

Kolejka priorytetowa w STL'u posiada następujące operacje:

- `top` - zwraca wartość pierwszego elementu w kolejce priorytetowej.
- `push` - umieszczenie nowego elementu na końcu kolejki priorytetowej;
- `pop` - usunięcie istniejącego elementu z początku kolejki priorytetowej;
- `empty` - informacja czy kolejka priorytetowa jest pusta;
- `size` - zwraca ilość elementów umieszczonych w kolejce priorytetowej;

Aby móc skorzystać z kolejki priorytetowej w C++, należy dołączyć plik nagłówkowy:

```
#include <queue>
```

Składnia

```
#include <queue>
using namespace std;

{
template <
class __Typ,
class __Kontener = vector < __Typ >,
class __Porzadek = less < typename __Kontener::value_type >
> class priority_queue;
}
```

Dostarcza interfejs kolejki priorytetowej dla kontenera przekazanego jako parametr szablonu __Kontener.

Kolejność porządkowa jest ustalana na podstawie parametru __Porzadek.

Złożoność obliczeniowa poszczególnych metod kolejki priorytetowej jest zależna od kontenera, który jest z nim skojarzony.

Przykład:

```
priority_queue < int > kolejka;     // domyślnie rosnąca
priority_queue < int, vector<int>, greater <int> > kolejka1;  // malejąco
priority_queue < int, vector<int>, less <int> > kolejka2;     // rosnaco
```

top

Zwraca referencję do najwyższego elementu w kolejce priorytetowej. Ten element zostanie usunięty po wywołaniu funkcji pop(). Jeśli używana jest domyślna funkcja porównująca, zwracany element jest również największy spośród elementów w kolejce.

Przykład 1

```
1  #include<iostream>
2  #include<queue>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  int tab[5]={1,2,3,4,5};
9  priority_queue < int > kolejka(tab, tab+5); // rosnąco
10
11 cout<<kolejka.top(); // 5
12
13 return 0;}
```

Przykład 2

```
1  #include<iostream>
2  #include<queue>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  priority_queue < int > kolejka;
9  kolejka.push(10);
10 kolejka.push(40);
11 cout<<kolejka.top(); // 40
12
13 return 0;}
```

push

Metoda push służy do umieszczenia nowego elementu w kolejce priorytetowej

```
1  #include<iostream>
2  #include<queue>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  priority_queue < int > kolejka;
9  kolejka.push(10);
10 kolejka.push(40);
11 kolejka.push(20);
12 kolejka.push(30);
13
14 // kolejka: 40 30 20 10
15
16 return 0;}
```

pop

Metoda pop służy do usunięcia istniejącego elementu z początku kolejki priorytetowej. Standardowo jest to wartość największa.

```
1  #include<iostream>
2  #include<queue>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  priority_queue < int > kolejka;
9  kolejka.push(10);
10 kolejka.push(40);
11 kolejka.pop(); // usunięcie 40
12 cout<<kolejka.top(); // 10
13
14 return 0;}
```

empty

Metoda empty sprawdza, czy kolejka priorytetowa jest pusta.

```
1  #include<iostream>
2  #include<queue>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  priority_queue < int > kolejka;
9  if(kolejka.empty()==true){cout<<"pusta"<<endl;} // pusta
10 kolejka.push(10);
11 kolejka.push(40);
12 if(kolejka.empty()==false){cout<<"pełna"<<endl;} // pełna
13
14 return 0;}
```

sice

Metoda size zwraca ilość elementów aktualnie znajdujących się w kolejce priorytetowej.

```
1  #include<iostream>
2  #include<queue>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  priority_queue < int > kolejka;
9  kolejka.push(10);
10 kolejka.push(40);
11 kolejka.push(30);
12 cout<<"rozmiar kolejki="<<kolejka.size();
13 // rozmiar kolejki=3
14 return 0;}
```

Ćwiczenie 1

Zapoznanie się z podstawowym interfejsem kolejki priorytetowej.

Utwórz program w C++, który:

1. Zadeklaruje kolejkę priorytetową liczb całkowitych (domyślne porządkowanie).
2. Wstawi do niej pięć dowolnych liczb.
3. Wyświetli element o najwyższym priorytecie za pomocą top().
4. Usunie jeden element z kolejki i ponownie wyświetli aktualny element najwyższego priorytetu.

Ćwiczenie 2

Praktyczne użycie metod empty() oraz size().

Napisz program, który:

1. Tworzy pustą kolejkę priorytetową typu int.
2. Sprawdza, czy kolejka jest pusta i wyświetla odpowiedni komunikat.
3. Dodaje do kolejki trzy elementy.
4. Wyświetla liczbę elementów w kolejce.
5. Usuwa wszystkie elementy w pętli, kontrolując warunek zakończenia za pomocą empty().

Ćwiczenie 3

Zastosowanie własnego kryterium porządkowania.

Zadeklaruj kolejkę priorytetową, w której:

1. Element o **najmniejszej wartości** ma najwyższy priorytet.
2. Wstaw do niej co najmniej sześć liczb całkowitych.
3. Wyświetl wszystkie elementy w kolejności ich usuwania z kolejki.

Wskazówka: Zastosuj greater<int> jako trzeci parametr szablonu.

Ćwiczenie 4

Praca ze złożonymi danymi.

1. Zdefiniuj strukturę Zadanie, zawierającą:
    a. nazwę zadania (string),
    b. priorytet (int).
2. Utwórz kolejkę priorytetową przechowującą obiekty typu Zadanie, uporządkowaną malejąco według priorytetu.
3. Dodaj kilka zadań o różnych priorytetach.
4. Wyświetl zadania w kolejności ich realizacji.

Ćwiczenie 5

Zastosowanie kolejki priorytetowej.

Zaimplementuj program symulujący system obsługi zgłoszeń technicznych:

1. Każde zgłoszenie zawiera: identyfikator, opis oraz priorytet.
2. Program umożliwia:
    a. dodanie nowego zgłoszenia,
    b. obsłużenie zgłoszenia o najwyższym priorytecie,
    c. wyświetlenie liczby oczekujących zgłoszeń.
3. Program działa w pętli do momentu wybrania opcji zakończenia.